

Model Deployment with FLASK API



Introduction



Deployment of machine learning application

Introduction

This lesson introduces the FLASK framework for deploying ML applications to servers. First, we start by creating a simple FLASK application that prints the message “Hello, World!” Then, the application is deployed locally on the laptop’s localhost server(not a live server).

Next, an ML model of Iris categorization is built and deployed on the localhost using FLASK. A light background in web development and knowledge of HTML tags will help, but it is unnecessary. In this lesson, we use the Windows Power Shell command line. However, little modifications can apply instructions to Linux or Mac environments.

Virtual Python Environment and install FLASK

Start by opening the PowerShell prompt on Windows. We will create a virtual environment and install all the needed packages in this environment. This way, the installed packages are removed from your system when the virtual environment is terminated.

Create a project director, and change to that directory:

```
C:\Users\amf> mkdir exampleFlask  
C:\Users\amf> cd exampleFlask
```

Ensure Python is installed. You can check this by issuing the following command. If not installed, then you must install it.

```
C:\Users\amf\exampleFlask> python --version
```

Create a virtual Python environment in this directory and activate it.

```
C:\Users\amf\exampleFlask> python -m venv env  
C:\Users\amf\exampleFlask> .\env\scripts\activate  
(env) PS C:\Users\amf\exampleFlask>
```

Next, install Flask using the command: `pip install flask`.

```
(env) C:\Users\amf\exampleFlask> pip install flask
```

Create a Flask application with the code below and save as “app.py”. Make sure the file is stored in exampleFlask directory.

```
from flask import Flask app = Flask( name ) @app.route('/')  
def hello():  
    return 'Hello, World!'  
if name == ' main ': app.run(debug=True)
```

Your first Flask application is now ready to run. Issue the following command:

```
(env) C:\Users\amf\exampleFlask> flask run
```

System will then return with the URI of where the application is posted. Copy/paste the URL in your browser and verify the message.

Ctrl-C key combination will terminate the application.



Next, we will utilize Git/GitHub tools and post the “Hello World!” Flask application on Heroku server (rather than our localhost).

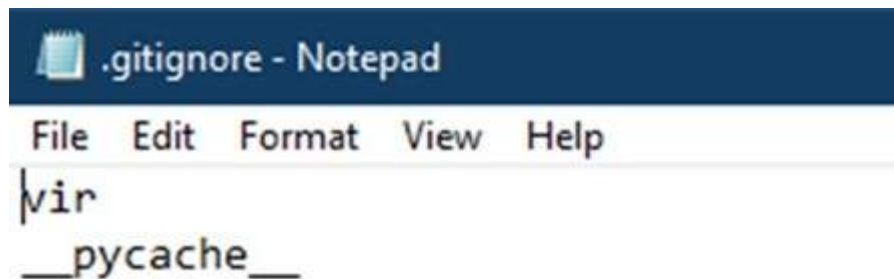
At the system prompt, issue the following command that turns the Heroku directory into a git director for tracking changes.

```
(env) PS C:\Users\amf\exampleFlask> git init
```

Next, we need to create two files:

1. A .gitignore file that lists the files and directories that should be ignored when updating things.
2. The file “Procfile” that is needed by the Heroku server.

Create the files with the content shown and save them in exampleFlask director.



Install gunicorn, which is the webserver to be used by Heroku. Also need the list of all the required packages to build this application in the requirements.txt file. Finally, the freeze command builds the requirements.txt file.

```
(env) PS C:\Users\amf\exampleFlask> pip install gunicorn
(env) PS C:\Users\amf\exampleFlask> pip freeze > requirements.txt *saves ne
(env) PS C:\Users\amf\exampleFlask> git add .gitignore app.py Procfile requ
(env) PS C:\Users\amf\exampleFlask> git commit -m "Initialize Git repositor
```

At the system prompt, download Heroku CLI (Command Line Interface) from the Heroku site:

<https://devcenter.heroku.com/articles/heroku-cli>

Create a free account at the Heroku site:

<https://www.heroku.com/home>

Login to Heroku using Heroku CLI. Issue the following command at the prompt.

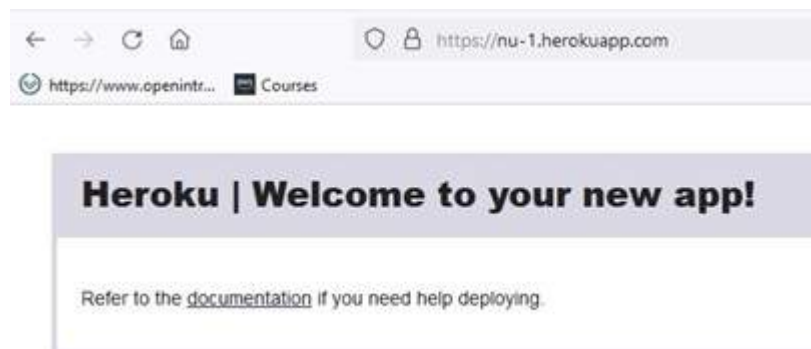
```
(env) C:\Users\amf\exampleFlask> heroku login
```

Next, create a Heroku application called nu-1.

```
(env) C:\Users\amf\exampleFlask> heroku create nu-1
```

The system responds by returning the URL of the Heroku server where the application is posted. Note that Heroku has its git repository as well.

```
Creating 2 nu-1... done
https://nu-1.herokuapp.com/ | https://git.heroku.com/nu-1.git
```



Next, push your github to Heroku github to see the application running on the Heroku server. Issue the command.

```
(env) PS C:\Users\amf\exampleFlask> git push https://git.heroku.com/nu-1.g:
```

Go to the Heroku site, and click on the dashboard. Next, select the nu-1 application from the list of personal applications. In the nu-1 application, click on setting and go

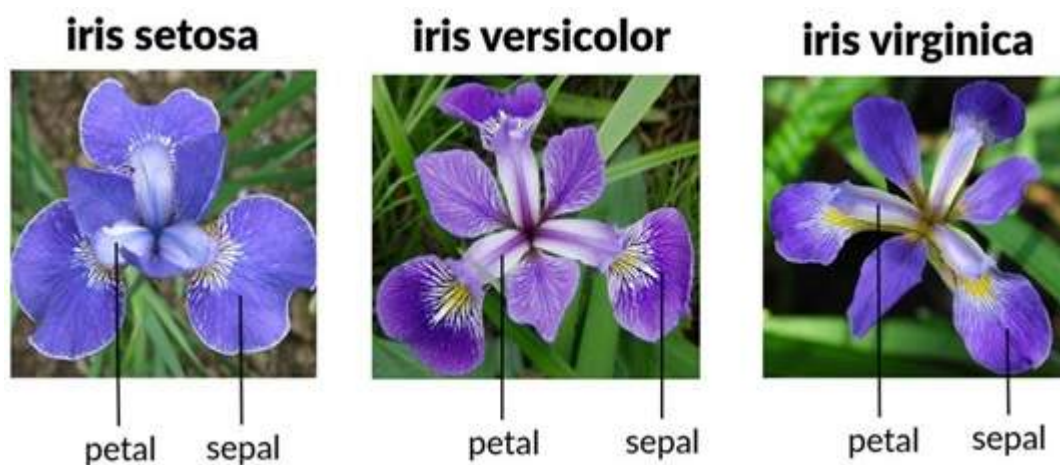
towards the end of the page to see the application page.

The screenshot displays the Heroku dashboard interface. On the left sidebar, the Heroku logo is at the top, followed by a 'Personal' dropdown menu. Below this is a search bar labeled 'Filter apps and pipelines'. A list of applications follows: 'ana680-3', 'firsttest20', and 'nu-1', which is highlighted with a yellow background. The main content area shows the 'nu-1' application page. At the top, there's a breadcrumb 'Personal > nu-1' and a navigation bar with tabs: 'Overview', 'Resources', 'Deploy', 'Metrics', 'Activity', 'Access', and 'Settings' (highlighted in yellow). The main text states 'Your app can be found at' followed by the URL 'https://nu-1.herokuapp.com/' which is also highlighted in yellow. Below this, a browser window is shown with the address bar containing the same URL. The browser's address bar also shows 'https://www.openintr...' and 'Courses'. The page content below the browser window displays 'Hello, World!'.

Deployment of machine learning application

Deploying ML on a local machine using Flask

Consider the Iris.csv dataset and the ML classification problem of identifying the type of Iris based on the width and length of petal and sepal of the flowers



There are a few different ways to start a Jupyter notebook. We can start Jupyter from Windows Powershell, Anaconda distribution, or by using Google Colab. In this example, we will use Windows PowerShell.

Open Windows PowerShell and check whether python is installed by issuing the command. If installed, the system returns the python's version number

```
PS C:\Users\amf> python
Python 3.9.7 (tags/v3.9.7:1016ef3, Aug 30 2021, 20:19:38) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()
PS C:\Users\amf>
```

If Python is not installed, download it from the site:

<https://www.python.org/downloads/>

and install it like other software installations on Windows.

With Python installed, we will next install pip. Navigate to the site:

<https://bootstrap.pypa.io/get-pip.py>

and the browser immediately prompts for downloading the file get-pip.py. Download the get-pip.py file and remember the location of the file. In this example, the file was copied to the current directory.

Navigate back to the PowerShell prompt and enter the command “python get-pip.py”

```
PS C:\Users\amf> python get-pip.py
Collecting pip
  Downloading pip-22.1-py3-none-any.whl (2.1 MB)
    ----- 2.1/2.1 MB 0.7 MB/s eta 0:00:00
Collecting wheel
  Downloading wheel-0.37.1-py2.py3-none-any.whl (35 kB)
Installing collected packages: wheel, pip
```

Check the version of pip installed by issuing the command pip -V at the system prompt.

```
PS C:\Users\amf> pip -V  
pip 22.1 from C:\Users\amf\
```

Now we can use pip to manage and install python packages.

The pip version can be upgraded by the following command at the prompt

```
python -m pip install --upgrade pip
```

Install Jupyter notebook by issuing the command: `pip install notebook`. To start the notebook, give the command: `jupyter notebook`.

To start the project, we go to the PowerShell prompt and follow the steps below :

1. Create a new directory called irisProject (or some other name): `mkdir irisProject`
2. Change directory to irisProject: `cd irisProject`
3. Create a virtual environment. Call it venviris
4. Activate the environment

```
PS C:\Users\amf> cd irisProject  
PS C:\Users\amf\irisProject> python -m venv venviris
```

```
PS C:\Users\amf\irisProject> .\venviris\scripts\activate
(venviris) PS C:\Users\amf\irisProject>
```

5. Initialize the directory as a git repository

```
(venviris) PS C:\Users\amf\irisProject> git init
Initialized empty Git repository in C:/Users/amf/irisProject/.git/
(venviris) PS C:\Users\amf\irisProject>
```

Finally, start the Jupyter notebook

```
(venviris) PS C:\Users\amf\irisProject> jupyter notebook
```

In the browser, open a new notebook.



Step 2

```
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
#data set has no column headings
col_name = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
df = pd.read_csv(url, names = col_name)
df.head()
```

Start by loading the data set in the Jupyter notebook and inspecting the top few observations

Step 3

```
df.shape #---> (150, 5)
df.info() #---> no missing data, dtype; float64 and on object
df.describe() #---> numerical values are in range of 0 to 8|
```

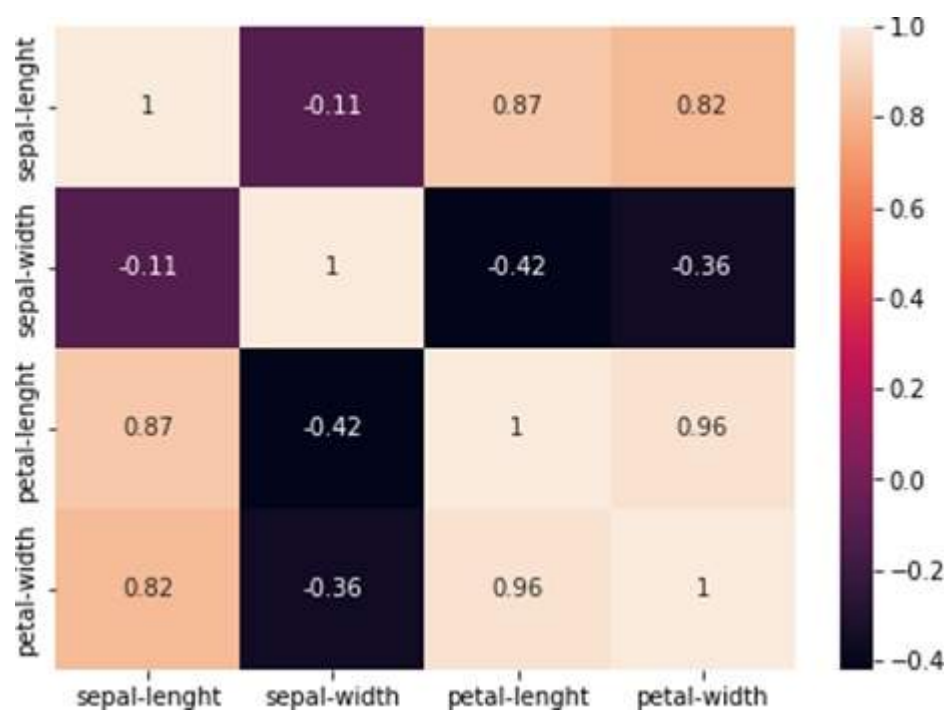
Explore the data set

Step 4

```
#heatmap
plt.figure(figsize=(7,5))
sns.heatmap(df.corr(), annot=True)
plt.show()
```

Complete tasks for data analysis and visualization

Step 5



Step 6

```
x = df.drop(['class'], axis=1) #X is having all the independent variables  
y = df['class'] ##Y is having an dependent variable 'Class'
```

Define the independent and dependent variables

Step 7

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=1)
```

Split dataset for the training and verification

Step 8

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=1)
```

Use Support Vector Machine (SVM) classifier. Algorithm for model training, producing 97% accuracy

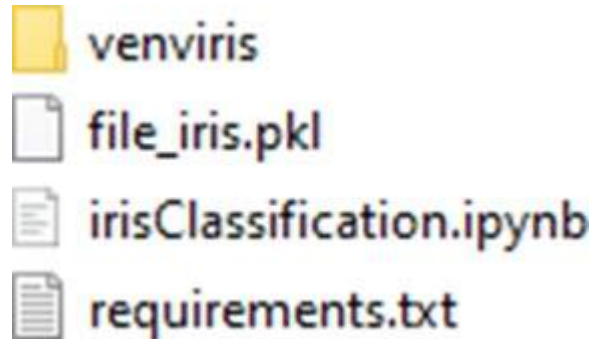
Step 9

```
import pickle
# Open a file and use dump()
with open('file_iris.pkl', 'wb') as file:

    # A new file will be created
    pickle.dump(model, file)
```

Next, save the model as a pickle format file. This file will be saved in the same location as the Jupyter notebook file. Finally, rename the Jupyter notebook as “irisClassification.ipynb”.

Step 9 Title



A requirement file contains all the libraries/packages used in building the project. To get the requirement file, we need to write a command in Windows PowerShell. Then, close Jupyter notebook and issue the command.

pip freeze>requirements.txt

After this command, the requirements.txt will be generated and saved in the same folder where the jupyter files and model files were saved.

Next, we need to create an HTML file for the client-side to receive input from the user. The user input is fed to the model to make predictions.

Create a folder called “templates” and save this HTML file as “index.html” in the templates folder.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>CreditCard</title>
```

```

</head>
<body>
<form action="{{ url_for('predict')}}" method="POST"> Sepal_Length: <input

<input type=submit      value=Submit>
</form>
My prediction: {{ predict }}
</body>
</html>

```

This webpage is displayed by the HTML code above. This page is presented to the user for input.

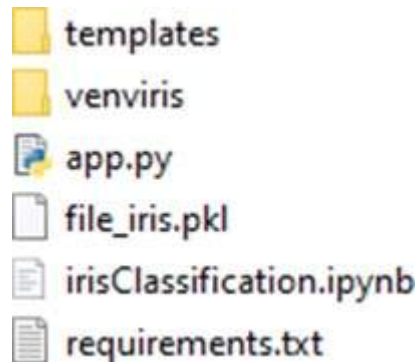
As a final step we also need our Flask/Python code to receive user input, run it through the model and return the predication. This file is saved as app.py

```

from flask import Flask, render_template, request import numpy as np
import pickle #import joblib
app = Flask(__name__) filename = 'file_iris.pkl'
model = pickle.load(open(filename, 'rb')) # load the model
#model = joblib.load(filename) # two ways to load the model, not using joblib
@app.route('/') def index():
return render_template('index.html')

```

```
@app.route('/predict', methods=['POST']) # The user input is processed here
Sepal_Length = request.form['sepal_length'] Sepal_Width = request.form['sepal_width']
pred = model.predict(np.array([[Sepal_Length, Sepal_Width, Petal_Length, Petal_Width]])
return render_template('index.html', predict=str(pred)) if __name__ == '__main__':
app.run(debug=True)
```



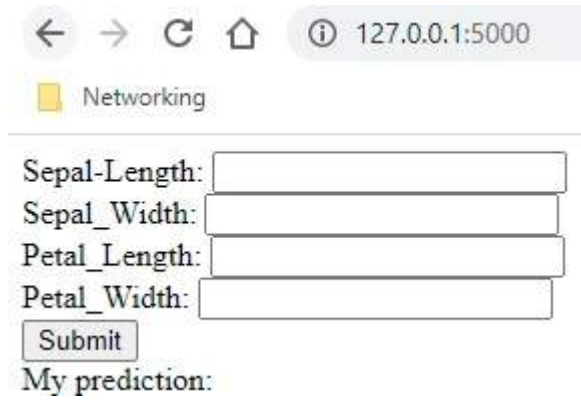
To test the local deployment, double-click on app.py (type flask run). This will bring up a message similar to the one in the image below and gives the localhost URL where the application runs.

```
* Serving Flask app 'app2' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Restarting with stat
* Debugger is active!
* Debugger PIN: 509-958-687
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```


Copy URL and paste it into the browser to see the applications

Step 2

Double click on app.py to start the application

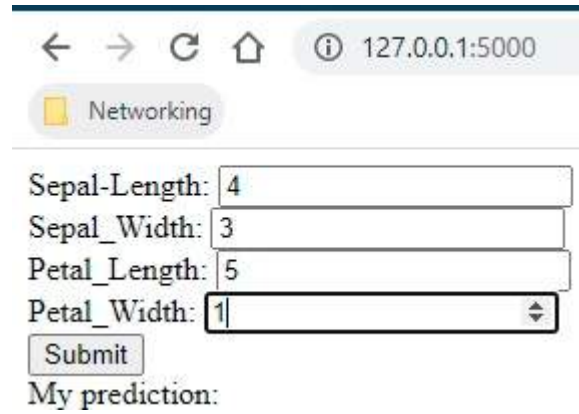


The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000". Below the address bar, there is a tab labeled "Networking". The main content area of the browser displays a web application with the following elements:

- Four input fields labeled "Sepal-Length:", "Sepal_Width:", "Petal_Length:", and "Petal_Width:" stacked vertically.
- A "Submit" button located below the input fields.
- A label "My prediction:" located below the "Submit" button.

Step 3

Enter data into the fields



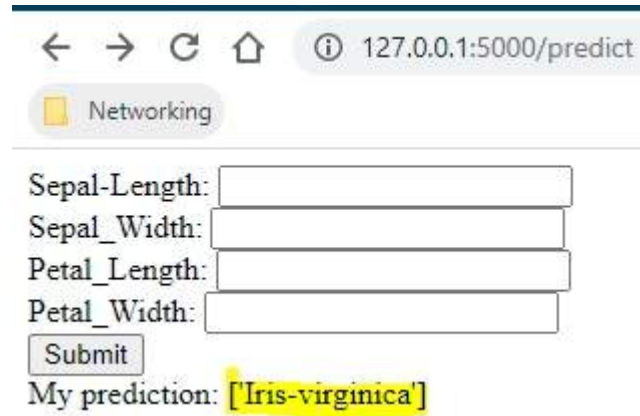
The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000". Below the address bar, there is a tab labeled "Networking". The main content area contains four input fields for data entry:

- Sepal-Length: 4
- Sepal_Width: 3
- Petal_Length: 5
- Petal_Width: 1

Below the input fields is a "Submit" button. At the bottom of the form, the text "My prediction:" is displayed.

Step 4

Click on the submit button to see the result



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000/predict". Below the address bar, there is a tab labeled "Networking". The main content area contains four input fields for "Sepal-Length:", "Sepal_Width:", "Petal_Length:", and "Petal_Width:". Below these fields is a "Submit" button. The output line "My prediction: ['Iris-virginica']" is displayed, with the prediction result highlighted in yellow.

← → ↻ 🏠 ⓘ 127.0.0.1:5000/predict

📁 Networking

Sepal-Length:

Sepal_Width:

Petal_Length:

Petal_Width:

My prediction: ['Iris-virginica']